

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8



Nguyễn Trí Dũng

CTO • Builder • AI-native Software Engineering

15+ năm xây dựng sản phẩm và đội ngũ kỹ thuật, tập trung vào các hệ thống phần mềm thực tế và AI ứng dụng.



EXPERIENCE

- 10+ năm làm việc & hợp tác với các đối tác tại Nhật Bản
TV Tokyo, Sony Music, DeNA, Kayac và nhiều dự án khác.
- Đồng sáng lập & CTO tại Fabbi
Xây dựng tổ chức kỹ thuật từ startup, phát triển các sản phẩm và giải pháp cho thị trường Việt Nam & Nhật Bản.



CURRENT FOCUS

- Reverse Engineering & Code Analysis
Phân tích mã nguồn, trích xuất kiến trúc, tạo tài liệu thiết kế, change tracking phục vụ modernize hệ thống legacy.
- Coding Agent Platform (Rust)
Agentic coding trên Web • Desktop • CLI.
- AI-native Engineering & Team Collaboration
Áp dụng AI (coding agent) hiệu quả trong đội ngũ kỹ thuật lớn.



COMMUNITY PROJECT

- Mazii.net – dự án giáo dục cộng đồng
Đồng sáng lập năm 2013. Mazii giúp học tiếng Nhật qua web & ứng dụng, hiện được Google xếp hạng #1 trong lĩnh vực học tiếng Nhật với hàng triệu lượt truy cập mỗi tháng.



SCAN TO
CONNECT
WITH ME



Facebook
facebook.com/dzungntnew



Website
dzungtri.github.io

“ Khi mọi người đều có coding agent và có thể vibe coding, chúng ta sẽ xây team, giao tiếp và phát triển sản phẩm như thế nào?”

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8

MỘT THAY ĐỔI CHƯA TỪNG CÓ

Mỗi engineer giờ đây đều có một “đội quân AI” riêng.

TRƯỚC ĐÂY

10 engineers

↓

10 người làm việc

BÂY GIỜ

100+ AI Agents

↓

110+ “workers”

Claude Code
Codex
Cursor
OpenCode
Antigravity
Custom Agents
MCP Tools
Debug AI
Refactor Agent

Vibe Coding
+ Multi Agents
+ Tools

Productivity của cá nhân tăng vọt.

Nhưng câu hỏi quan trọng hơn là...

? Nếu mỗi người đều có một “đội quân AI” riêng, thì team sẽ vận hành như thế nào? ?



- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8

KHI THÊM AI VÀO TEAM...

AI = Robot xây cực nhanh ⚡

NHANH HƠN RẤT NHIỀU ⚡

AI không có context tổng thể.
Nó chỉ tối ưu task cục bộ.

AI TĂNG TỐC ĐỘ,
NHƯNG KHÔNG THAY ĐỔI QUY LUẬT.

NHƯNG...

- ❌ KHÔNG CÓ BẢN VẼ
→ XÂY SAI
- ❌ KHÔNG THEO CHUẨN
→ LỆCH HỆ THỐNG
- ❌ KHÔNG PHỐI HỢP
→ PHÁ CÔNG TRÌNH

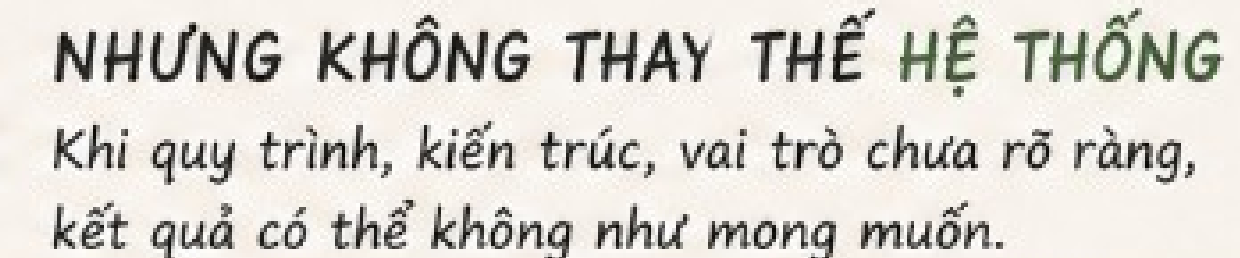
★ Không có quy trình & kiến trúc → AI không giúp bạn xây nhanh, nó giúp bạn phá nhanh hơn.

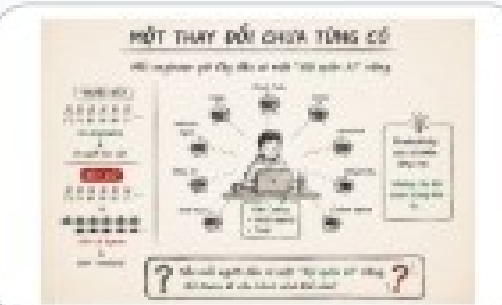
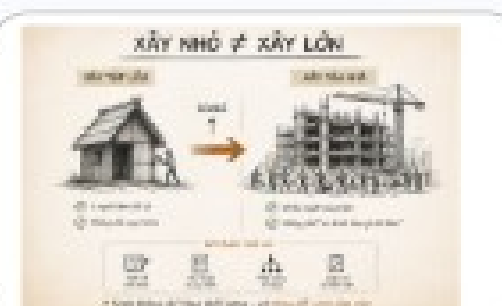
[<](#)

[>](#)



Cần team

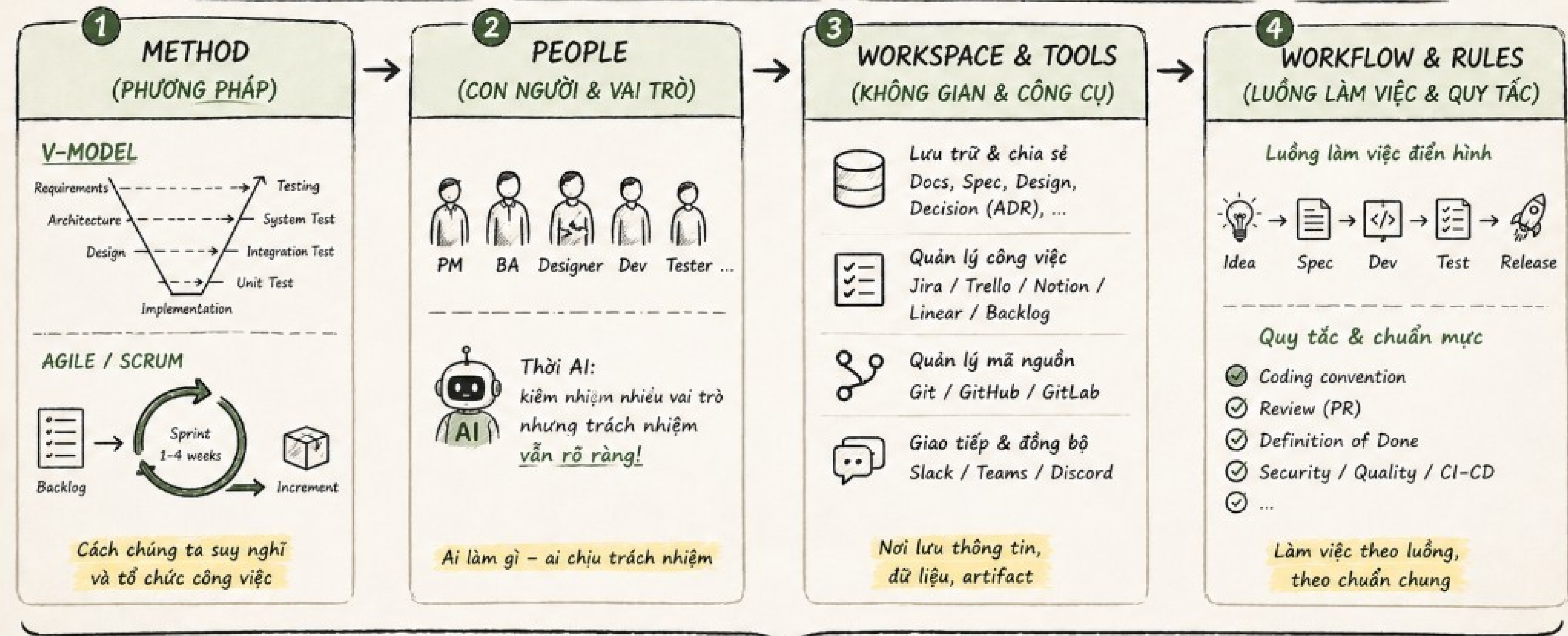
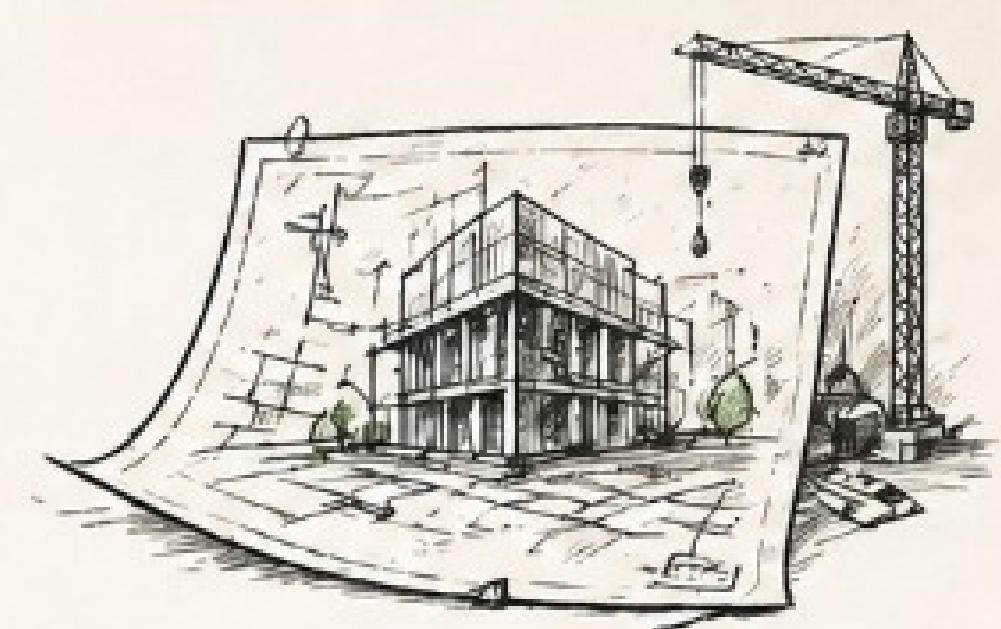


- 1 
- 2 
- 3 
- 4 
- 5 
- 6 
- 7 
- 8 

4/- QUY TRÌNH PHÁT TRIỂN PHẦN MỀM TRÔNG NHƯ THỂ NÀO?



Quy trình không chỉ là vài bước - đó là một hệ thống vận hành.



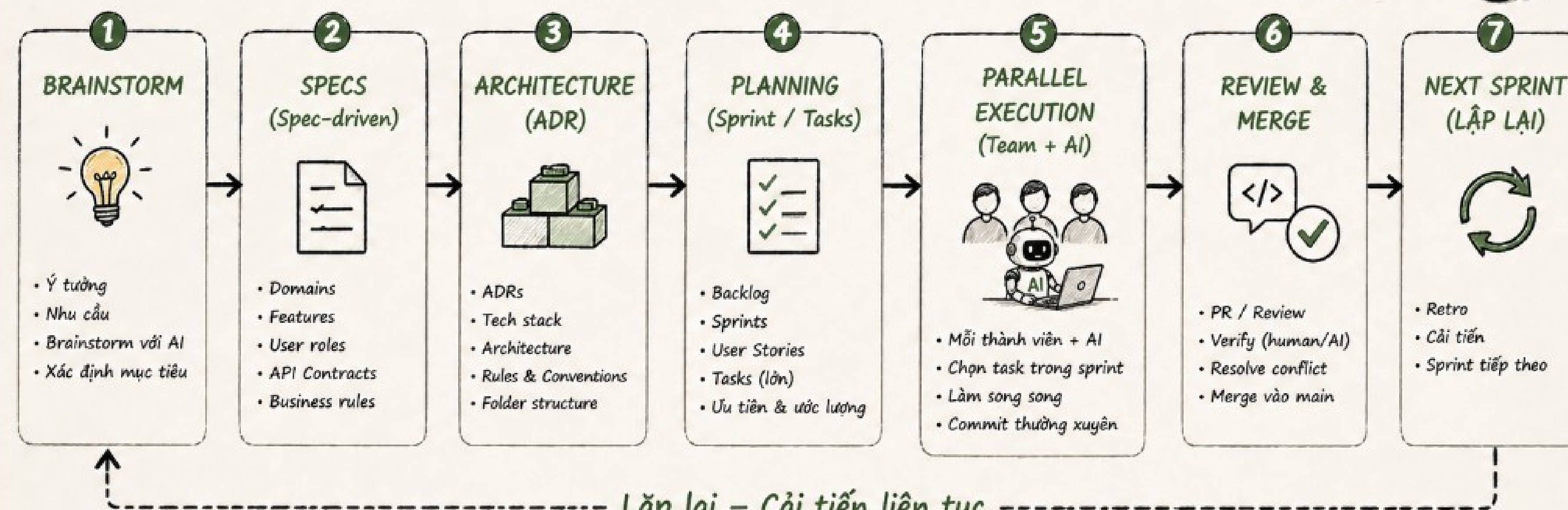
Quy trình = **HỆ THỐNG** vận hành đầy đủ

★ Thiếu 1 trong 4 yếu tố, team vẫn chạy nhưng dễ bị lệch, chậm, và tốn kém.



Một team + AI vận hành dự án như thế nào?

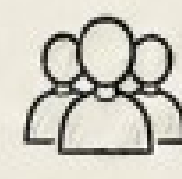
Từ ý tưởng đến sản phẩm - lặp lại theo Sprint.



NỀN TẢNG CHUNG



WORKSPACE
Git Repository
(Shared for all)



ROLES & RESPONSIBILITIES
Rõ ràng, linh hoạt
(Human + AI)



TOOLS
Git, CI/CD, Test,
Docs, AI Agents,
MCP (Jira, Linear,)



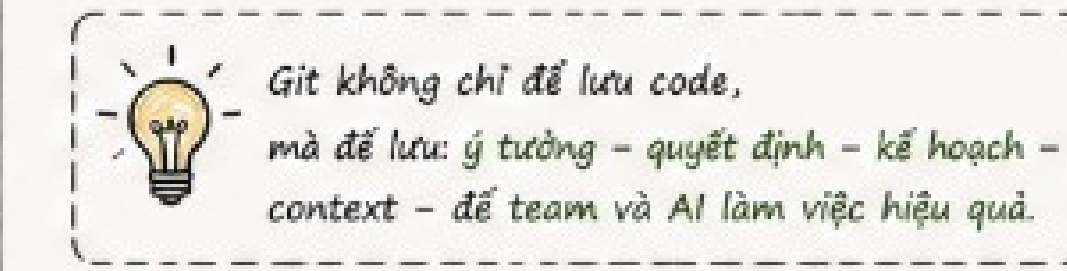
**RULES &
STANDARDS**
Coding convention,
Definition of Done,
Review rules...

Mọi hoạt động đều dựa trên context chung: Specs + ADRs + Contracts + Rules.
AI mạnh khi context rõ ràng!





Shared workspace cho cả team + AI: cùng đọc - cùng hiểu - cùng cập nhật



	/specs Nơi chứa đặc tả nghiệp vụ, domain, tính năng, API contracts, ...	
	/adrs Ghi lại các quyết định kiến trúc, công nghệ, quy ước, nguyên tắc.	
	/planning Kế hoạch sản phẩm, backlog, sprint plan, timeline, ...	
	/tasks Danh sách user stories / tasks theo Sprint, trạng thái, owner, ...	
	/src Codebase của dự án.	
	/docs Tài liệu hướng dẫn, runbook, how-to, architecture diagrams, ...	

- 3
- 4
- 5
- 6
- 7
- 8
- 9
-
-
-

3/4 THỰC THI: TEAM + AI LÀM VIỆC MỖI NGÀY

Triển khai song song - dựa trên context chung - theo Sprint - theo Workflow chuẩn

💡 Mục tiêu: Mỗi thành viên + AI hoàn thành một Task lớn trong Sprint
→ Đóng góp qua PR - Review - Merge vào main

TEAM ROLES

PM
Định hướng
Ưu tiên
Quản lý Sprint

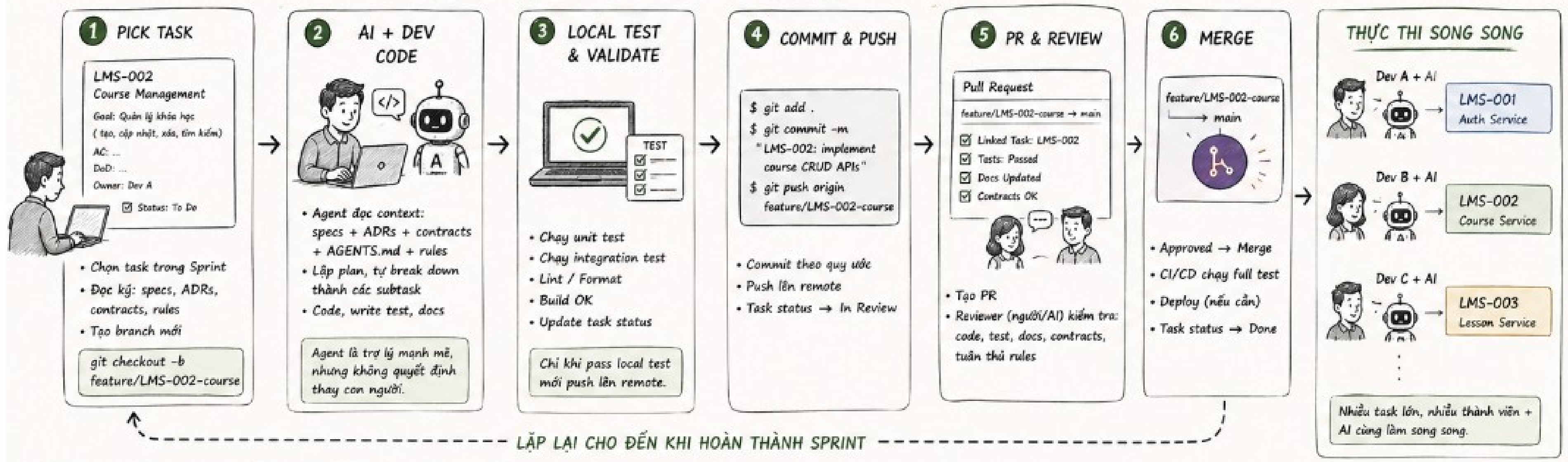
Architect
Kiến trúc
ADR
Tech decisions

Dev
Code
Tích hợp
Refactor

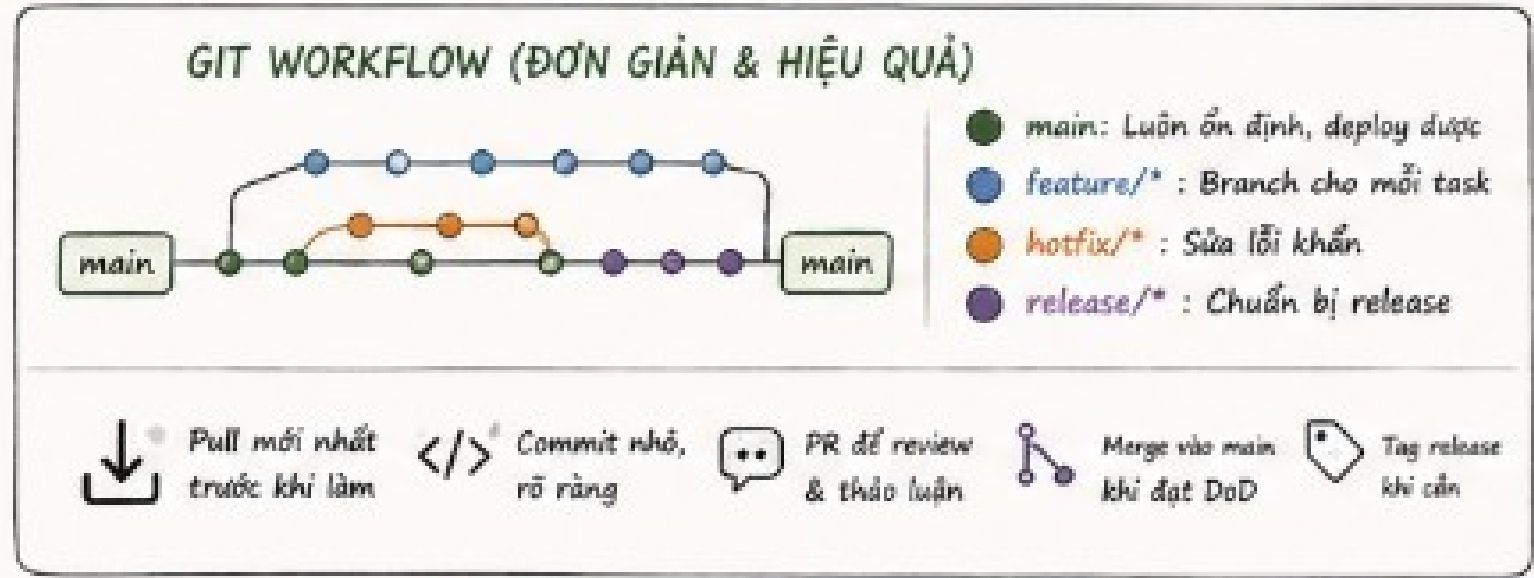
QA/Tester
Test
Automation
Quality gate

Ops/DevOps
CI/CD
Deploy
Monitoring

MỖI TASK LỚN TRONG SPRINT



----- LẶP LẠI CHO ĐẾN KHI HOÀN THÀNH SPRINT -----



AGENT ALWAYS READS (CONTEXT BASE)

- /specs → Sản phẩm & nghiệp vụ
- /adrs → Quyết định kiến trúc & kỹ thuật
- /contracts → API / gRPC / Event contracts
- /tasks/{id}.md → Mục tiêu, AC, DoD, liên kết
- /AGENTS.md → Quy tắc & workflow của dự án
- /src → Codebase

Context càng rõ ràng → Agent càng làm đúng hướng → ít vòng lặp.

DEFINITION OF DONE (MỖI TASK)

- ✓ Code hoàn thành theo specs
- ✓ Test pass (unit + integration)
- ✓ Docs / Contracts cập nhật (nếu có)
- ✓ Tuân thủ ADRs & coding rules
- ✓ PR được review & merge
- ✓ Task status = Done

Không merge khi chưa đạt DoD.

★ Workflow rõ ràng + Context đầy đủ + Review chặt chẽ → Code chất lượng + Tiến độ ổn định + Team scale được với AI.

- 4
- 5
- 6
- 7
- 8
- 9
- 10

4/4

NGUYÊN TẮC VÀ LƯU Ý QUAN TRỌNG ĐỂ TEAM + AI LÀM VIỆC HIỆU QUẢ

Context đúng → AI hiểu đúng → Code đúng → Sản phẩm đúng

NHỚ:
"AI không thay thế team.
AI khuếch đại cách team
làm việc!"

1 SPEC-DRIVEN
(KHÔNG CODE TRƯỚC)

- Làm rõ "cái gì", "tại sao", "như thế nào" trước khi code.
- Specs là nguồn sự thật chung cho cả team + AI.

Spec tốt = ít đổi hướng = ít tốn chi phí

2 ADR LÀ NỀN
(KHÔNG MẠNH AI NÀO MỘT KIỂU)

- Mọi quyết định kiến trúc, công nghệ, quy tắc... đều ghi lại.
- Thay đổi phải có ADR mới.

ADR giúp team thống nhất và dễ dàng scale.

3 TASK ĐỦ LỚN
(AI TỰ CHIA NHỎ)

- Team chỉ quản lý task lớn (feature level).
- AI có không gian để tự lập plan và chia subtasks.

Đừng chia quá nhỏ, hãy để AI phát huy.

4 LÀM VIỆC QUA CONTRACT
(KHÔNG PHỤ THUỘC NGẮM)

Contract / API

- Giao tiếp giữa các domain/service qua contract (API/Proto/Events).
- Team chỉ cần biết "cam kết", không cần biết "cách làm".

Contract rõ ràng = tách rời tốt = làm song song hiệu quả

5 AI LUÔN ĐỌC CONTEXT
(TRƯỚC KHI CODE)

- Agent phải đọc đầy đủ context base trước khi bắt đầu.
- Cập nhật context là trách nhiệm của cả team.

Context càng rõ ràng AI càng làm đúng hướng.

6 GIT WORKFLOW LÀ KỶ LUẬT

- Branch rõ ràng.
- PR + Review bắt buộc.
- Mọi thay đổi đều có lịch sử và thể truy vết.

Kỷ luật Git = an toàn = dễ cộng tác

CONTEXT BASE PHẢI LUÔN ĐƯỢC DUY TRÌ

Specs cập nhật khi có thay đổi

ADR luôn mới nhất

Contract/Docs luôn đúng

Tasks & plan cập nhật trạng thái

AGENTS.md luôn rõ ràng

→

KẾT QUẢ BẠN NHẬN ĐƯỢC

- Team cùng hiểu, cùng một hướng
- AI làm việc như một thành viên thực thụ
- Code chất lượng, ít lỗi, ít rework
- Tiến độ ổn định và dự đoán được
- Dễ dàng on-boarding thành viên mới

★ TƯ DUY THEN CHỐT: Xây dựng context ngay từ đầu – Duy trì kỷ luật – Cải tiến liên tục → TEAM + AI = TĂNG TỐC BỀN VỮNG

TIP: Nếu team cảm thấy "AI làm ra kết quả không đúng mong muốn" → rất có thể là do context chưa đủ tốt hoặc chưa được cập nhật!

- 5
- 6
- 7
- 8
- 9
- 10
- 11

BONUS 1/2

BONUS: ADR TỐT GIÚP TEAM + AI LÀM VIỆC HIỆU QUẢ (1/2)

ADR không chỉ là “ghi lại quyết định” – mà là tạo ra bối cảnh đúng để team và AI làm việc song song.

1

ADR: TEST-DRIVEN / TEST STRATEGY

QUYẾT ĐỊNH

Team ưu tiên viết test (unit/integration/e2e) hoặc acceptance criteria rõ ràng trước khi implementation.

VÌ SAO THÂN THIỆN VỚI AI

✓ Agent có “đích đến” rõ ràng.

✓ Agent có thể tự code → tự chạy test → tự sửa.

✓ Review dễ hơn vì nhìn vào test pass/fail.

CÂU CHỐT: Test là hợp đồng hành vi của hệ thống.

2

ADR: DOMAIN SPLIT (MICROSERVICE / HYBRID)

QUYẾT ĐỊNH

Chia hệ thống theo domain rõ ràng. Có thể Microservice hoặc Modular Monolith (hybrid).

VÌ SAO THÂN THIỆN VỚI TEAM + AI

✓ Mỗi member phụ trách một domain → làm song song dễ hơn.

✓ Giảm phụ thuộc, ít conflict code.

✓ Agent chỉ cần context của service/module đó → giảm “over-context”.

CÂU CHỐT: Chia domain tốt = chia context tốt.

3

ADR: API CONTRACT FIRST

QUYẾT ĐỊNH

Các service giao tiếp qua contract rõ ràng trước khi code:

OpenAPI (REST)

gRPC Proto

Event Schema

DTO Schema

...

LỢI ÍCH

✓ Service A không cần đọc sâu code Service B.

✓ Frontend, Backend, Agent có thể làm song song.

✓ Khi contract đổi → phải review rõ ràng.

CÂU CHỐT: Team scale bằng contract, không scale bằng hiểu ngầm.

3 ADR này giúp: phân ranh giới rõ – giảm phụ thuộc – chạy song song – AI hiểu đúng & làm đúng.

BONUS 2/2

BONUS: ADR TỐT GIÚP TEAM + AI LÀM VIỆC HIỆU QUẢ (2/2)

4

ADR: PROJECT STRUCTURE & CODING CONVENTION

QUYẾT ĐỊNH

Thống nhất cấu trúc dự án, quy ước code và kỹ thuật.

QUY ƯỚC BAO GỒM

VÌ SAO QUAN TRỌNG VỚI AI

• Agent không tự sáng tạo mỗi lần một kiểu.

• Code sinh ra đồng nhất hơn.

• Review, bảo trì, refactor dễ dàng hơn.

CÂU CHỐT: Không có convention, AI sẽ nhân bản sự hỗn loạn rất nhanh.

5

ADR: DEFINITION OF DONE (DOD)

QUYẾT ĐỊNH

Một task chỉ được đóng khi đáp ứng đầy đủ:

LỢI ÍCH VỚI TEAM + AI

✓ Agent biết khi nào được dùng.

✓ Member biết khi nào được merge.

✓ Tránh tình trạng “code xong rồi nhưng chưa xong thật”.

CÂU CHỐT: Done không phải là code chạy được. Done là đủ chuẩn để merge.

TÓM TẮT TƯ DUY

CHIA RÕ (Boundary rõ)

↓

CONTRACT RÕ (Giao tiếp rõ)

↓

TEST RÕ (Đích đến rõ)

↓

AI + TEAM CHẠY SONG SONG HIỆU QUẢ

ADR tốt = Context đủ – Boundary rõ – Contract rõ – Test rõ → Team + AI làm việc song song, chất lượng cao và bền vững.